Выполнение лабораторной работы №2

Первоначальная настройка git.

Карпухин Клим

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

4.1 Установка программного обеспечения 8
4.2 Базовая настройка Git 8
4.3 Создание SSH-ключа 9
4.4 Создание PGP-ключа 9
4.5 Настройка Github 10
4.6 Добавление PGP ключа в Github 11
4.7 Настройка автоматических подписей коммитов git 12
4.8 Настройка gh 12
4.9 Шаблон для рабочего пространства 13
4.10 Настройка каталога курса 14

5 Контрольные вопросы **15**

6 Выводы **17**

Список литературы **18**

Список иллюстраций

4.1 Установка git и gh 8
4.2 Установка имени и email репозитория, настройка utf-8 в выводе сообщений git, установка имени начальной ветки, настройка параметры autocrlf и safecrlf 8
4.3 Генерация ключа по алгоритму rsa с ключём размером 4096 бит . 9
4.4 Генерация ключа по алгоритму ed25519 9
4.5 Создание ключа PGP 10
4.6 Демострация учётной записи Github 11
4.7 Вывод списка ключей 11
4.8 Вывод списка ключей 11
4.9 Установка полученного gpg ключа 12
4.10 Настройка автоматических прописей гит 12
4.11 Процесс авторизации 13
4.12 Создания репозитория курса на основе шаблона 13
4.13 Настройка каталога курса 1 14
4.14 Настройка каталога курса 2 14

Список таблиц

1 Цель работы

- Изучить идеологиию и применение средств контроля версий.
- Освоить умения по работе с git

2 Задание

- Создать базовую конфигурацию для работы с git.
- Создать ключ SSH.
- Создать ключ PGP.
- Настроить подписи git.
- Зарегистрироваться на Github.
- Создать локальный каталог для выполнения заданий по предмету.

3 Теоретическое введение

Git - распределённая система контроля версий, предназначенная для отслеживания изменений в файлах и организации совместной работы над проектами. Для безопасного подключения к удалённым репозиториям, например на Github, используются SSH-ключи, обеспечивающие аутентификацию без ввода пароля. Для подтверждения авторства коммитов применяются PGP-ключи (GPG), позволяющие подписывать изменения и делать их проверяемыми. Регистрация на Github даёт возможность удалённого хранения репозиториев и взаимодействия с другими разработчиками. Локальный каталог для выполнения заданий инициализируется как Git-репозиторий, в котором фиксируются все изменения в ходе работы.

4 Выполнение лабораторной работы

4.1 Установка программного обеспечения

Установил git и gh(рис. 4.1).

Рисунок 4.1: Установка git и gh

4.2 Базовая настройка Git

Задал имя и email репозитория, настрорил utf-8 в выводе сообщений git, задал имя начальной ветки, настроил параметры autocrlf и safecrlf (рис. 4.2).

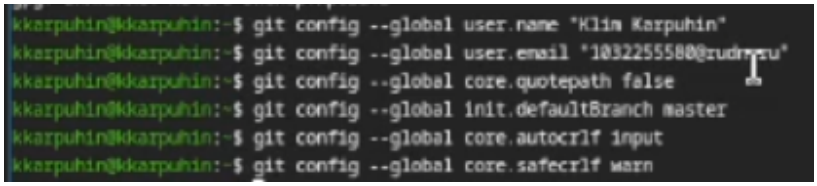

Рисунок 4.2: Установка имени и email репозитория, настройка utf-8 в выводе сообщений git, установка имени начальной ветки, настройка параметры autocrlf и safecrlf

4.3 Создание SSH-ключа

Сгенерировал ключ по алгоритму rsa с ключём размером 4096 бит(рис. 4.3).

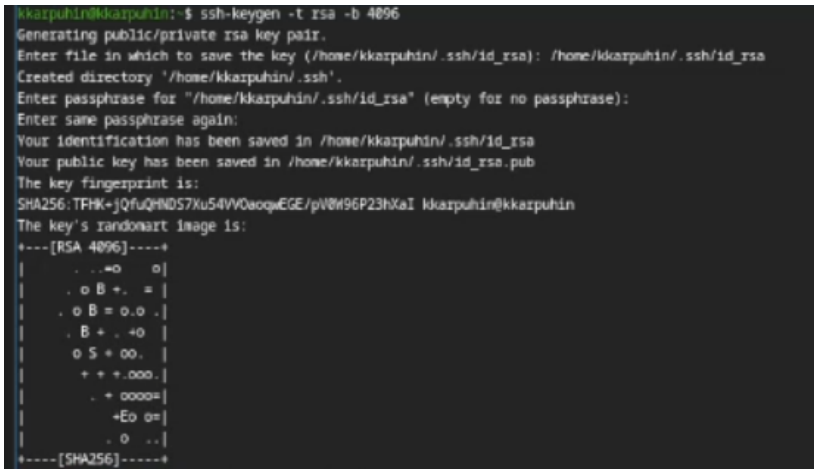

Рисунок 4.3: Генерация ключа по алгоритму rsa с ключём размером 4096 бит

Сгенерировал ключ по алгоритму ed25519(рис. 4.4).

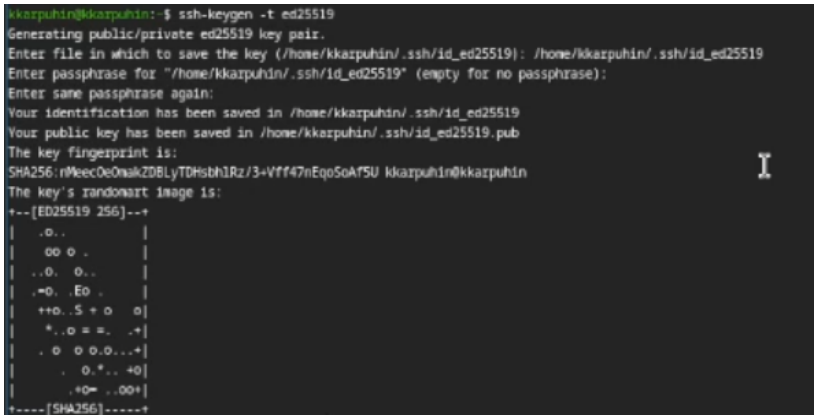

Рисунок 4.4: Генерация ключа по алгоритму ed25519

4.4 Создание PGP-ключа

Создал ключ PGP(рис. 4.5).

```
kkarpuhin@kkarpuhin:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Выберите тип ключа:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (только для подписи)
  (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
         0 = не ограничен
      <n>  = срок действия ключа - n дней
      <n>w = срок действия ключа - n недель
      <n>m = срок действия ключа - n месяцев
      <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Klim
Адрес электронной почты: kreazot318@gmail.com
Примечание:
Вы выбрали следующий идентификатор пользователя:
    "Klim <kreazot318@gmail.com>"
```

Рисунок 4.5: Создание ключа PGP

4.5 Настройка Github

Я уже создавал учётную запись Github ранее. В последующих лабараторных работах я продолжу пользоваться ею.(рис. 4.6).

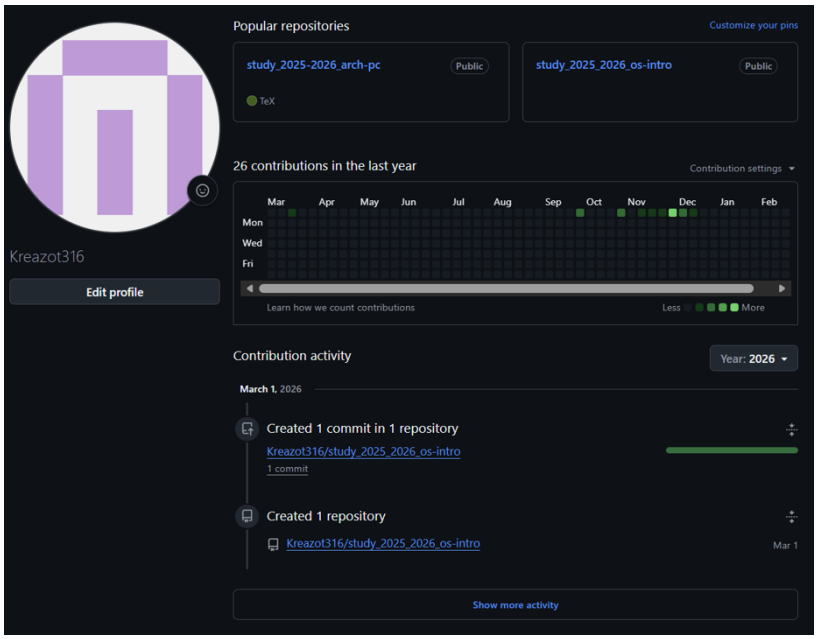

Рисунок 4.6: Демострация учётной записи Github

4.6 Добавление PGP ключа в Github

Вывел список ключей (рис. 4.7).

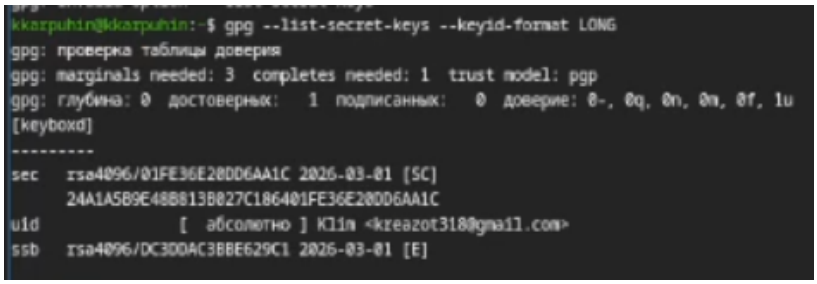

Рисунок 4.7: Вывод списка ключей

Скопировал сгенерированный GPG ключ в буфер обмена (рис. 4.8).

Рисунок 4.8: Вывод списка ключей

Перешёл в Github, нажал на кнопку *New GPG key* и вставил полученный ключ

в поле ввода(рис. 4.9).

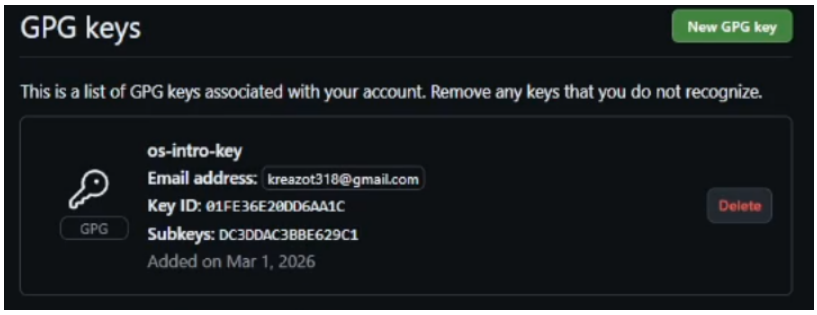

Рисунок 4.9: Установка полученного gpg ключа

4.7 Настройка автоматических подписей коммитов git

Используя введённый email, указал Git применять его при прописи коммитов(рис. 4.10).

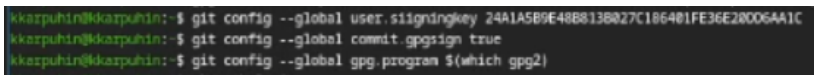

Рисунок 4.10: Настройка автоматических прописей гит

4.8 Настройка gh

Авторизация `gh auth login` (рис. 4.11).

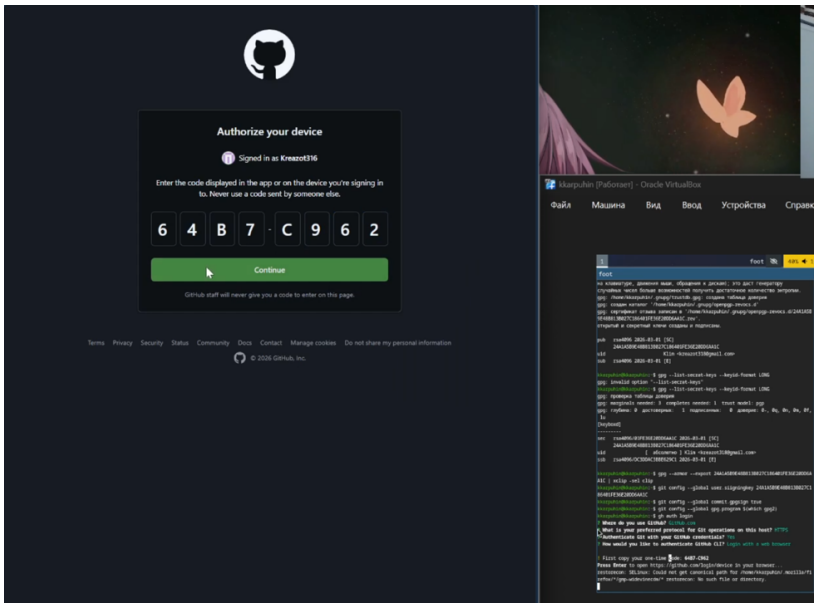

Рисунок 4.11: Процесс авторизации

4.9 Шаблон для рабочего пространства

Создал репозиторий курса на основе шаблона(рис. 4.12).

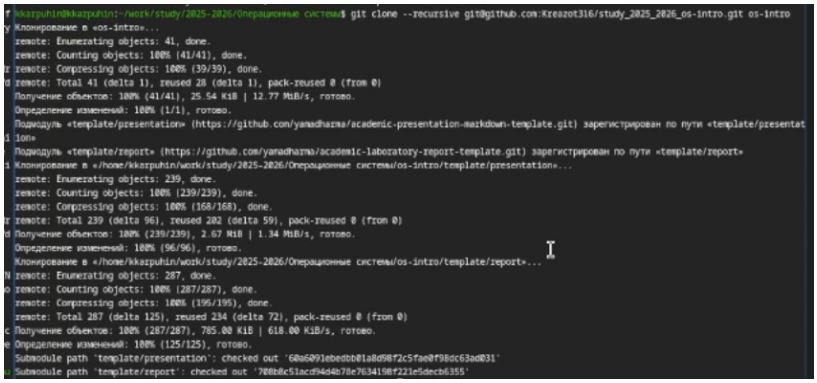

Рисунок 4.12: Создания репозитория курса на основе шаблона

4.10 Настройка каталога курса

Перешёл в каталог курса, удалил лишние файлы, создал необходимые каталоги(рис. 4.13).

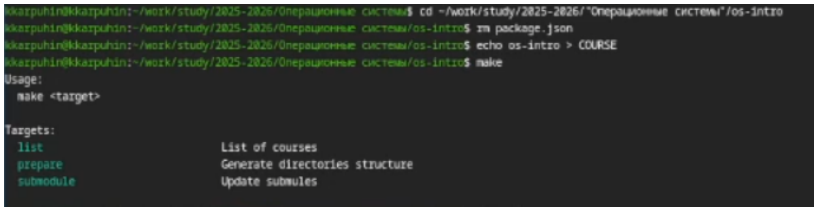

Рисунок 4.13: Настройка каталога курса 1

Отправил файлы на сервер(рис. 4.14).

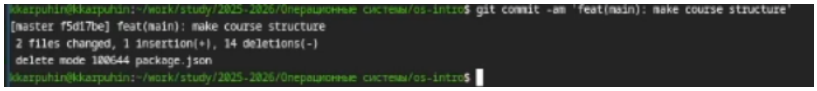

Рисунок 4.14: Настройка каталога курса 2

5 Контрольные вопросы

1. **Системы контроля версий (VCS)** — это инструменты, отслеживающие изменения в файлах. Они решают задачи сохранения истории проекта, возможности отката к старым верс>
2. **Хранилище** — это база данных проекта со всей историей. **Коммит** — сохраненный снимок состояния файлов в этот момент. **История** — последовательность таких сним>
3. В **централизованных** VCS (например, SVN) история хранится на одном сервере. В **децентрализованных** (Git, Mercurial) каждый разработчик имеет полную копию истории>
4. При единоличной работе вы инициализируете репозиторий, вносите изменения, добавляете их в индекс (add), фиксируете (`commit`) и при необходимости просматриваете ис>
5. При работе с общим хранилищем сначала получают актуальные изменения (`pull`), затем фиксируют свои наработки локально (`commit`), отправляют их на сервер (`push`) и >
6. Git управляет версиями, позволяет создавать и сливать ветки, гарантирует целостность данных (через хэширование) и обеспечивает высокую скорость работы благодаря лока>
7. Основные команды: `init` (создать), `clone` (копировать), add (подготовить), `commit` (сохранить), `status` (проверить), `log` (история), `pull` (забрать), `push` >
8. Локально: `git init` → `git add` → `git commit`. С удаленным: `git clone` →

`git push` (отправка) и `git pull` (получение обновлений).

9. **Ветки** — это отдельные линии разработки. Они нужны, чтобы параллельно вести новые функции или исправления ошибок, не затрагивая стабильную версию проекта до их по>
10. Для игнорирования файлов (например, временных, с секретами или зависимостями) используется файл `.gitignore`. Это делается, чтобы не засорять репозиторий мусором и не публиковать конфидециальные данные.

6 Выводы

В ходе лабораторной работы была выполнена базовая настройка Git, сгенерированны SSH и PGP ключи для аутентификации и подписи коммитов, а также была произведена регистрация и настройка учётной записи на Github. В результате настроено автоматическое подписание коммитов, что обеспечивает верификацию авторства, и созданно рабочее пространство с использованием шаблона репозитория, подготовленное для дальнейшего выполнения заданий по предмету.

Список литературы